RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ROJAA.DR

Academic Year: 2026-27 **Semester:** III **Branch:** BE-CSE-'H'

Register No. 211625070160 8

Certified that this is the bonafide record of work done by the above student in the CS23334 - Fundamentals of Data Science ***Laboratory during the academic year 2026 – 2027.***

3/11/26

Signature of Faculty in-charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

```
import pandas as pd
import matplotlib.pyplot as plt

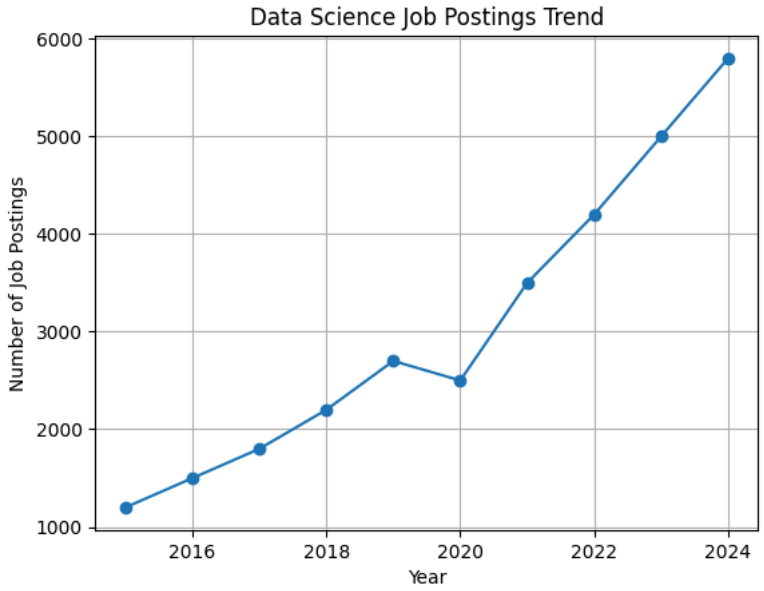
data = {
    'Year': [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024],
    'Job_Postings': [1200, 1500, 1800, 2200, 2700, 2500, 3500, 4200, 5000, 5800]
}

df = pd.DataFrame(data)

print(df)

plt.plot(df['Year'], df['Job_Postings'], marker='o')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.title('Data Science Job Postings Trend')
plt.grid(True)
plt.show()
```

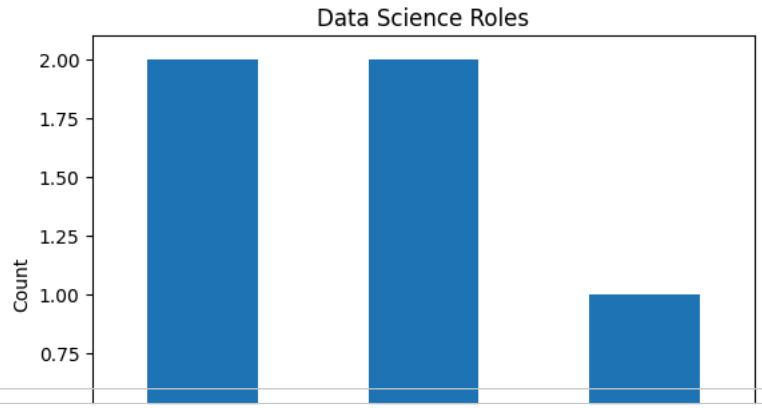
```
   Year  Job_Postings
0  2015          1200
1  2016          1500
2  2017          1800
3  2018          2200
4  2019          2700
5  2020          2500
6  2021          3500
7  2022          4200
8  2023          5000
9  2024          5800
```




```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'Role':['Data Analyst','Data Scientist','Data Engineer','Data Analyst','Data Scientist']
})

df['Role'].value_counts().plot(kind='bar')
plt.xlabel('Role')
plt.ylabel('Count')
plt.title('Data Science Roles')
plt.show()
```




```
import pandas as pd

structured = pd.DataFrame({
    'Name':['A','B','C'],
    'Age':[20,21,19]
})

semi = {'Name':'A', 'Skills':['Python','SQL']}

unstructured = "Data Science is the study of data."

print("Structured:\n", structured)
print("\nSemi-structured:\n", semi)
print("\nUnstructured:\n", unstructured)
```

```
Structured:
   Name  Age
0    A   20
1    B   21
2    C   19

Semi-structured:
 {'Name': 'A', 'Skills': ['Python', 'SQL']}

Unstructured:
 Data Science is the study of data.
```

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()
f = Fernet(key)

data = "Sensitive Data"
encrypted = f.encrypt(data.encode())
decrypted = f.decrypt(encrypted).decode()

print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```

```
Encrypted: b'gAAAAABqqg10Vjmp-myYzIwcXAf9ihBot74D-t-I0c_DHm99Tz6HloVQQvyT_5iTljetDSJLEc3lQ2oYhcKVn1Nj3iB8-83MDQ=='
Decrypted: Sensitive Data
```

Exp.no : 02 **ANALYSING AND VISUALISATION**

AIM:

Upload and analyse the given dataset and perform data preprocessing and visualisation.

PYTHON CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_excel("sales_data.xlsx")
print(df.head())
print(df.info())
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
plt.figure(figsize=(8, 5))
plt.scatter(
    df["Quantity"],
    df["Sales"],
    color="royalblue",

    edgecolors="black"
)
plt.title("Sales vs Quantity")
plt.xlabel("Quantity")
plt.ylabel("Sales")
plt.grid(True, alpha=0.3)
plt.show()

plt.hist(
    df["Sales"],
    bins=6,
    color="royalblue",
)
plt.title("Distribution of Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.grid(axis="y", alpha=0.3)
plt.show()
df.boxplot(
    column="Sales",
    by="Product",
    patch_artist=True,
)
plt.title("Sales Distribution by Product")
plt.suptitle("")
plt.xlabel("Product")
```

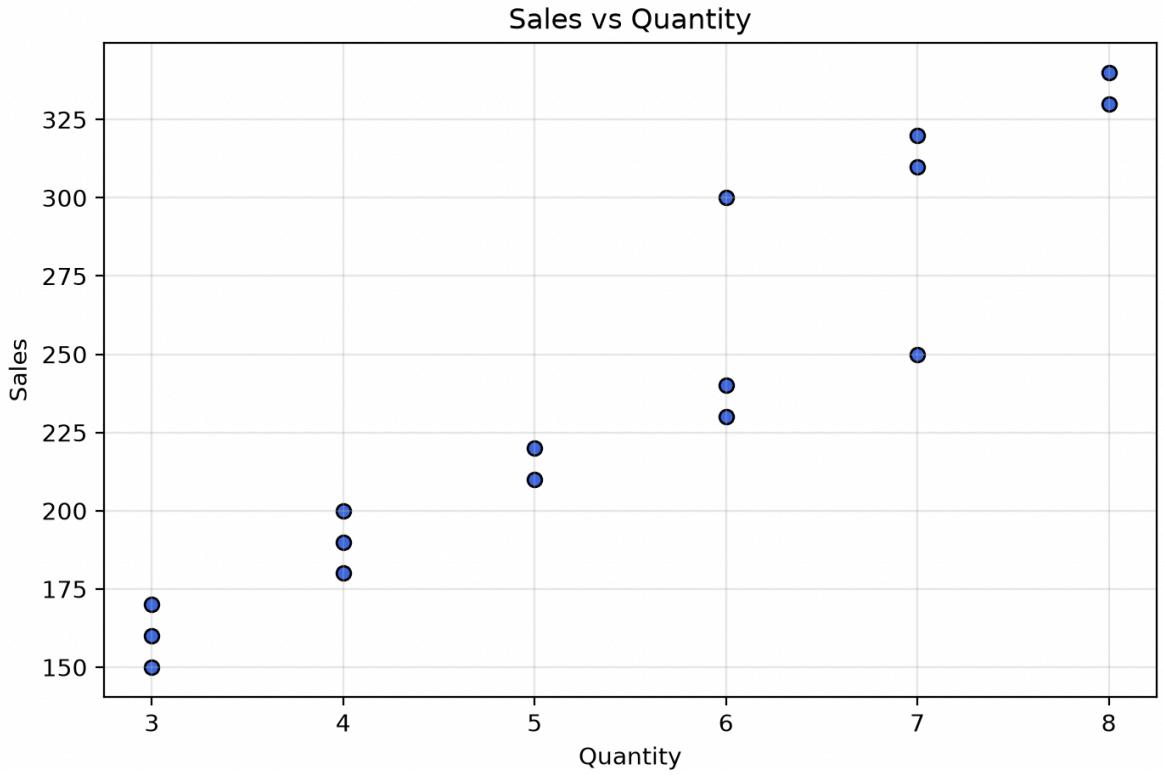
```
plt.ylabel("Sales")
plt.show()
```

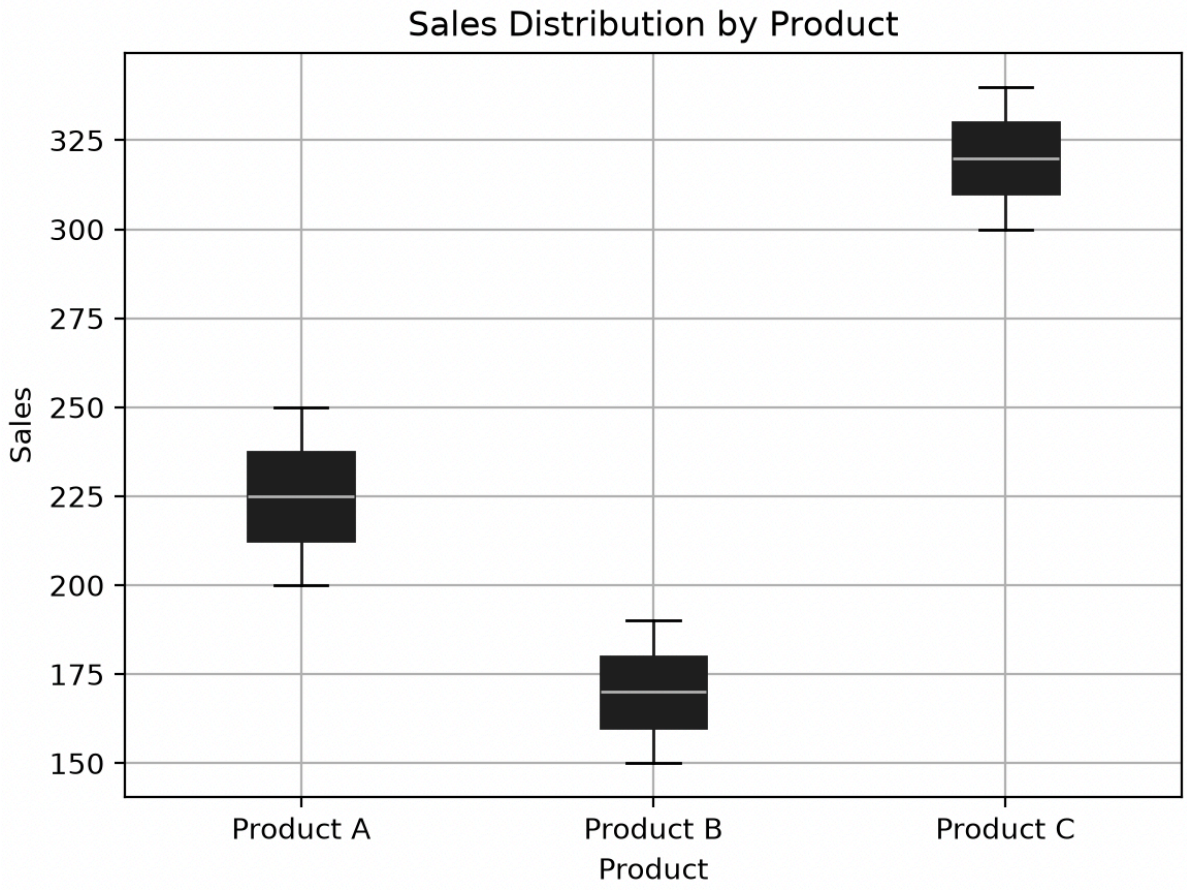
OUTPUT:

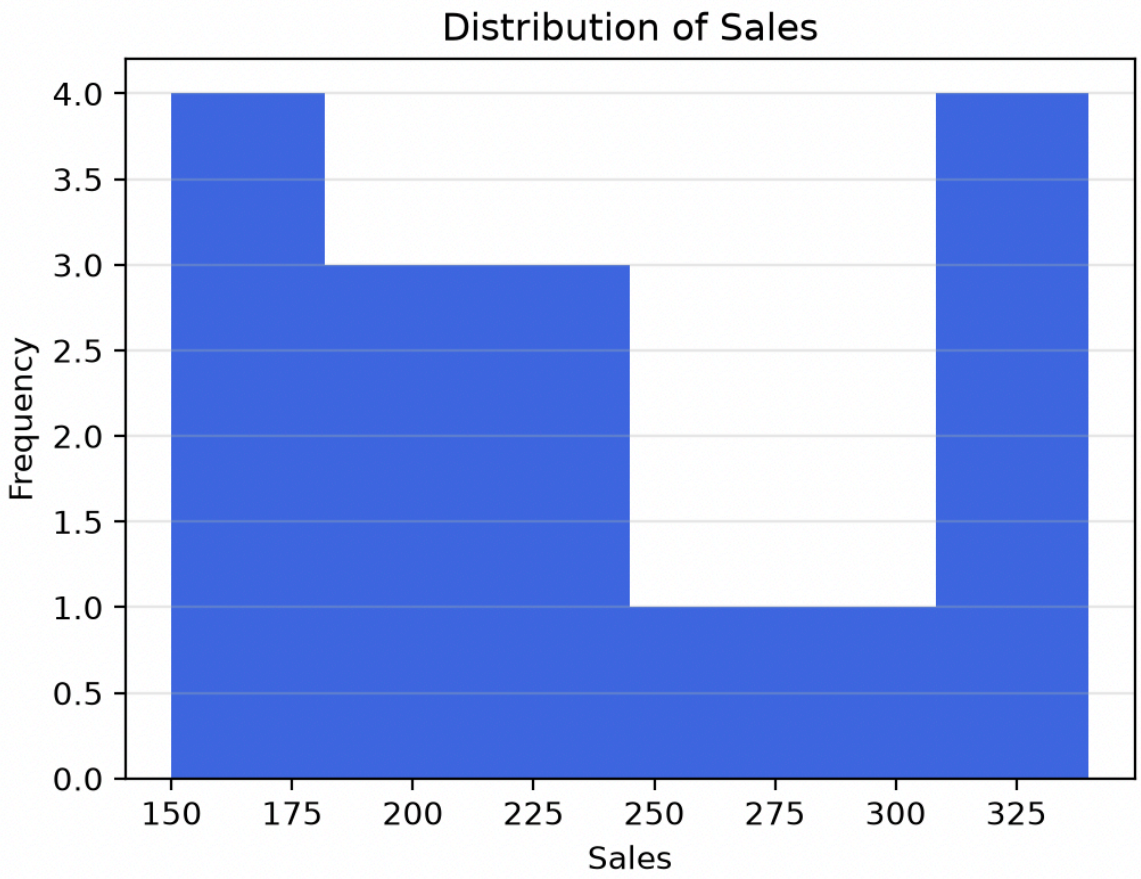
```
                 Date    Product  Sales  Quantity Region
0  2023-01-01 00:00:00  Product A    200         4  North
1  2023-02-01 00:00:00  Product B    150         3  South
2  2023-03-01 00:00:00  Product A    220         5  North
3  2023-04-01 00:00:00  Product C    300         6   East
4  2023-05-01 00:00:00  Product B    180         4   West
<class 'pandas.DataFrame'>
RangeIndex: 16 entries, 0 to 15
Data columns (total 5 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Date      16 non-null     object
 1   Product   16 non-null     str
 2   Sales     16 non-null     int64
 3   Quantity  16 non-null     int64
 4   Region    16 non-null     str
dtypes: int64(2), object(1), str(2)
memory usage: 772.0+ bytes
None

Missing Values:
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64

Duplicate Rows:
0
```







RESULT :

The code is successfully executed.

Exp.no : 03 **HANDLING MISSING AND INAPPROPRIATE DATA**

AIM:

Upload and analyse the given dataset and perform data preprocessing and visualisation.

PYTHON CODE:

```
import pandas as pd
import numpy as np

df = pd.read_excel("Hotel_Dataset.xlsx")

print("Original Dataset:")
print(df.head())

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Records:")
print(df.duplicated().sum())

df.drop_duplicates(inplace=True)

numeric_columns = df.select_dtypes(include=np.number).columns

for column in numeric_columns:
    df[column] = df[column].fillna(df[column].median())

categorical_columns = df.select_dtypes(include="object").columns

for column in categorical_columns:
    if df[column].isnull().any():
        df[column] = df[column].fillna(df[column].mode()[0])

for column in numeric_columns:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    df[column] = df[column].clip(lower_limit, upper_limit)

print("\nMissing Values After Preprocessing:")
print(df.isnull().sum())
```

```
print("\nDuplicate Records After Preprocessing:")
print(df.duplicated().sum())

print("\nPreprocessed Dataset:")
print(df.head())

df.to_excel("Preprocessed_Hotel_Dataset.xlsx", index=False)

print("\nData preprocessing completed successfully.")
```

OUTPUT:

Original Dataset:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+

Dataset Information:

```
<class 'pandas.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   CustomerID      11 non-null     int64
 1   Age_Group       11 non-null     str
 2   Rating(1-5)     11 non-null     int64
 3   Hotel           11 non-null     str
 4   FoodPreference  11 non-null     str
 5   Bill            11 non-null     int64
 6   NoOfPax         11 non-null     int64
 7   EstimatedSalary 11 non-null     int64
 8   Age_Group.1     11 non-null     str
dtypes: int64(5), str(4)
memory usage: 924.0 bytes
```

Missing Values:

```
CustomerID        0
Age_Group         0
Rating(1-5)       0
Hotel             0
FoodPreference    0
Bill              0
NoOfPax           0
```

```
EstimatedSalary    0
Age_Group.1        0
dtype: int64

Duplicate Records:
1
/Users/yuva/Desktop/Code_Hub/FDS_Exp/Exp_3/exp.py:25: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to writecode that works with pandas 2 and 3.
  categorical_columns = df.select_dtypes(include="object").columns

Missing Values After Preprocessing:
CustomerID         0
Age_Group          0
Rating(1-5)        0
Hotel              0
FoodPreference     0
Bill               0
NoOfPax            0
EstimatedSalary    0
Age_Group.1        0
dtype: int64

Duplicate Records After Preprocessing:
0

Preprocessed Dataset:
   CustomerID Age_Group  Rating(1-5)     Hotel FoodPreference    Bill  NoOfPax  EstimatedSalary Age_Group.1
0           1     20-25           4      Ibis           veg  1300.0      2.0          40000.0       20-25
1           2     30-35           5  LemonTree       Non-Veg  2000.0      3.0          59000.0       30-35
2           3     25-30           6    RedFox           Veg  1322.0      2.0          30000.0       25-30
3           4     20-25           0  LemonTree           Veg  1234.0      2.0         120000.0       20-25
4           5       35+           3      Ibis     Vegetarian   989.0      2.0          45000.0         35+

Data preprocessing completed successfully.
```

RESULT:

The given dataset is analysed and preprocessed.

Exp.no : 04

DATA PREPROCESSING

AIM:

Upload and analyse the given dataset and perform data preprocessing

PYTHON CODE:

```
import pandas as pd
import numpy as np

df = pd.read_excel("preprocessed_dataset.xlsx")

print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Records:")
print(df.duplicated().sum())

df.drop_duplicates(inplace=True)

numerical_columns = df.select_dtypes(include=np.number).columns
categorical_columns = df.select_dtypes(include="object").columns

for column in numerical_columns:
    df[column] = df[column].fillna(df[column].mean())

for column in categorical_columns:
    if df[column].isnull().any():
        df[column] = df[column].fillna(df[column].mode()[0])

print("\nMissing Values After Preprocessing:")
print(df.isnull().sum())

print("\nCleaned Dataset:")
print(df.head())

df.to_excel("preprocessed_dataset.xlsx", index=False)
```

OUTPUT:

```
 CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  EstimatedSalary
Age_Group.1
0          1     20-25           4       Ibis           veg  1300        2           40000       20-25
1          2     30-35           5  LemonTree        Non-Veg  2000        3           59000       30-35
2          3     25-30           6     RedFox           Veg  1322        2           30000       25-30
3          4     20-25          -1  LemonTree           Veg  1234        2          120000       20-25
4          5      35+           3       Ibis     Vegetarian   989        2           45000        35+

Dataset Shape:
(10, 9)

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       10 non-null     int64
 1   Age_Group        10 non-null     str
 2   Rating(1-5)      10 non-null     int64
 3   Hotel            10 non-null     str
 4   FoodPreference   10 non-null     str
 5   Bill             10 non-null     int64
 6   NoOfPax          10 non-null     int64
 7   EstimatedSalary  10 non-null     int64
 8   Age_Group.1      10 non-null     str
dtypes: int64(5), str(4)
memory usage: 852.0 bytes

Missing Values:
CustomerID         0
Age_Group          0
Rating(1-5)        0
Hotel              0
FoodPreference     0
Bill               0
NoOfPax            0
EstimatedSalary    0
Age_Group.1        0
dtype: int64

Duplicate Records:
0
/Users/yuva/Desktop/Code_Hub/FDS_Exp/Exp_3/exp4.py:23: Pandas4Warning: For backward
compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This
```

behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to writecode that works with pandas 2 and 3.
  categorical_columns = df.select_dtypes(include="object").columns

Missing Values After Preprocessing:
CustomerID 0
Age_Group 0
Rating(1-5) 0
Hotel 0
FoodPreference 0
Bill 0
NoOfPax 0
EstimatedSalary 0
Age_Group.1 0
dtype: int64

Cleaned Dataset:

	CustomerID	Age_Group	Rating(1-5)	Hotel	FoodPreference	Bill	NoOfPax	EstimatedSalary	Age_Group.1
0	1	20-25	4	Ibis	veg	1300	2	40000	20-25
1	2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
2	3	25-30	6	RedFox	Veg	1322	2	30000	25-30
3	4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
4	5	35+	3	Ibis	Vegetarian	989	2	45000	35+

RESULT:

The given dataset is analysed and preprocessed successfully.

Exp.no : 05

OUTLIERS

AIM:

Upload and analyse the given dataset and perform data preprocessing techniques and visualisation to find the outliers.

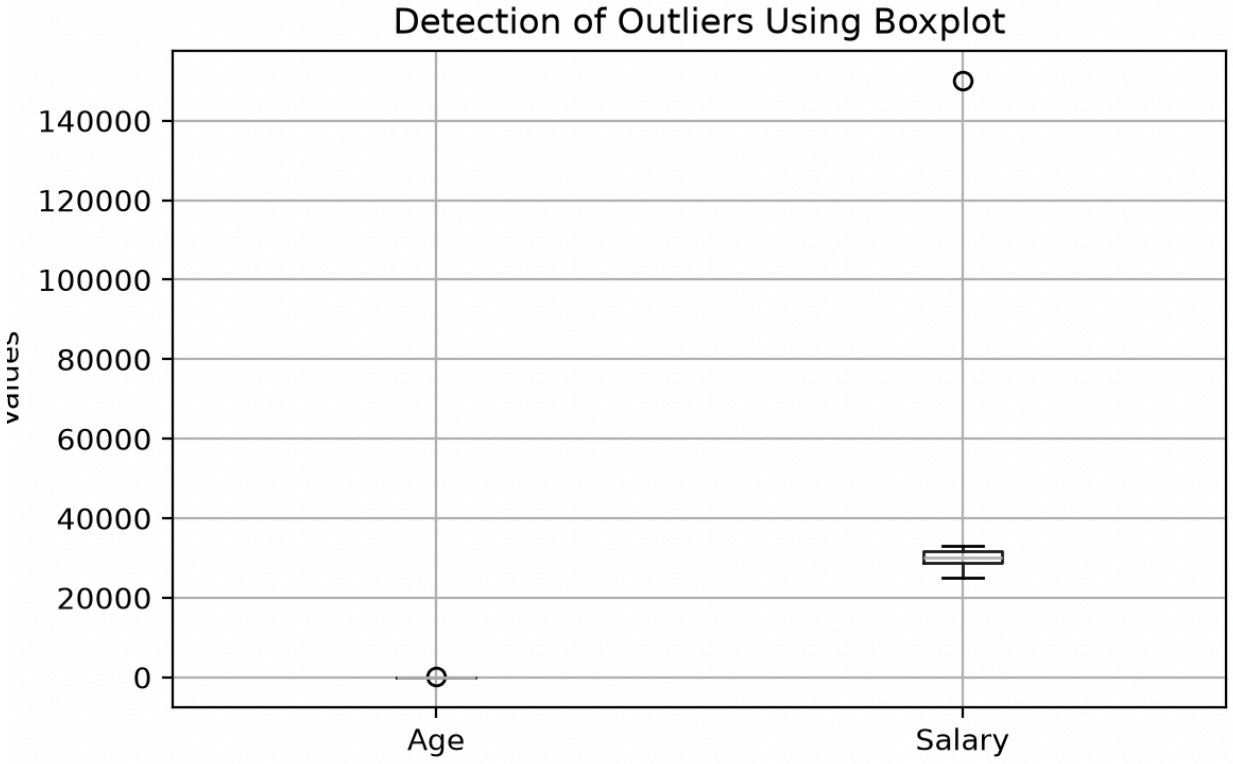
PYTHON CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
data = {
    "Age": [20, 21, 22, 23, 21, 24, 22, 25, 23, 22, 21, 100],
    "Salary": [25000, 28000, 30000, 32000, 29000, 31000,
               30500, 33000, 31500, 29500, 28500, 150000]
}
df = pd.DataFrame(data)
for column in df.select_dtypes(include="number").columns:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = df[
        (df[column] < lower_bound) |
        (df[column] > upper_bound)
    ]
    print("\nColumn:", column)
    print("Q1:", Q1)
    print("Q3:", Q3)
    print("IQR:", IQR)
    print("Lower Bound:", lower_bound)
    print("Upper Bound:", upper_bound)
    print("Outliers:")
    print(outliers[[column]])
plt.figure(figsize=(8, 5))
df.boxplot(column=["Age", "Salary"])
plt.title("Detection of Outliers Using Boxplot")
plt.ylabel("Values")
plt.show()
```

OUTPUT:

```
Column: Age
Q1: 21.0
Q3: 23.25
IQR: 2.25
Lower Bound: 17.625
Upper Bound: 26.625
Outliers:
    Age
11  100

Column: Salary
Q1: 28875.0
Q3: 31625.0
IQR: 2750.0
Lower Bound: 24750.0
Upper Bound: 35750.0
Outliers:
    Salary
11  150000
```



RESULT:

The given dataset is analysed and preprocessed successfully.

Exp.no : 06

FEATURE SCALING

AIM:

Upload and analyse the given dataset and perform data preprocessing techniques and feature scaling.

PYTHON CODE:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler

data = {
    'Country': ['France', 'Spain', 'Germany', 'Spain', 'Germany',
                'France', 'Spain', 'France', None, 'France'],
    'Age': [44.0, 27.0, 30.0, 38.0, 40.0,
            35.0, None, 48.0, 50.0, 37.0],
    'Salary': [72000.0, 48000.0, 54000.0, 61000.0, None,
               58000.0, 52000.0, 79000.0, 83000.0, 67000.0],
    'Purchased': ['No', 'Yes', 'No', 'No', 'Yes',
                  'Yes', 'No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())

X = df[['Age', 'Salary']]

standard_scaler = StandardScaler()
X_standardized = standard_scaler.fit_transform(X)

standardized_df = pd.DataFrame(
    X_standardized,
    columns=['Age', 'Salary']
)

print("\nStandardized Data:")
print(standardized_df)

minmax_scaler = MinMaxScaler()
X_normalized = minmax_scaler.fit_transform(X)

normalized_df = pd.DataFrame(
    X_normalized,
    columns=['Age', 'Salary']
)
print("\nNormalized Data:")
print(normalized_df)
```

OUTPUT:

Standardized Data:

	Age	Salary
0	0.758874	7.494733e-01
1	-1.711504	-1.438178e+00
2	-1.275555	-8.912655e-01
3	-0.113024	-2.532004e-01
4	0.177609	6.632192e-16
5	-0.548973	-5.266569e-01
6	0.000000	-1.073570e+00
7	1.340140	1.387538e+00
8	1.630773	1.752147e+00
9	-0.258340	2.937125e-01

Normalized Data:

	Age	Salary
0	0.739130	0.685714
1	0.000000	0.000000
2	0.130435	0.171429
3	0.478261	0.371429
4	0.565217	0.450794
5	0.347826	0.285714
6	0.512077	0.114286
7	0.913043	0.885714
8	1.000000	1.000000
9	0.434783	0.542857

RESULT:

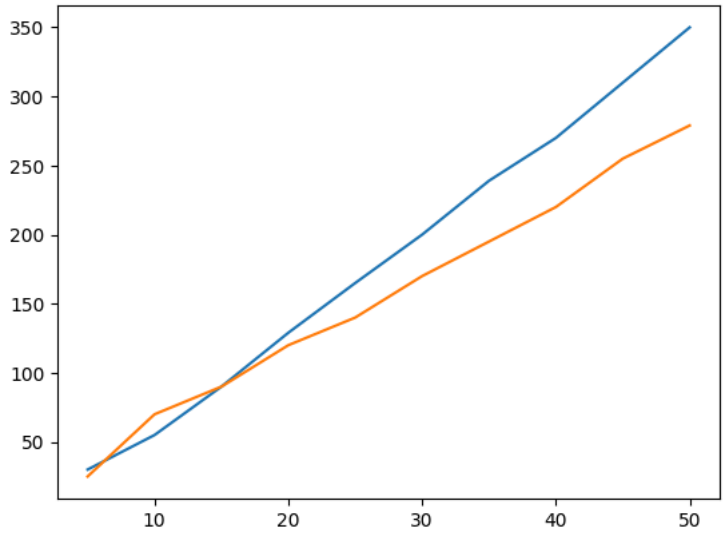
The given dataset is analysed and preprocessed successfully.

```
from google.colab import files
uploaded = files.upload()
```

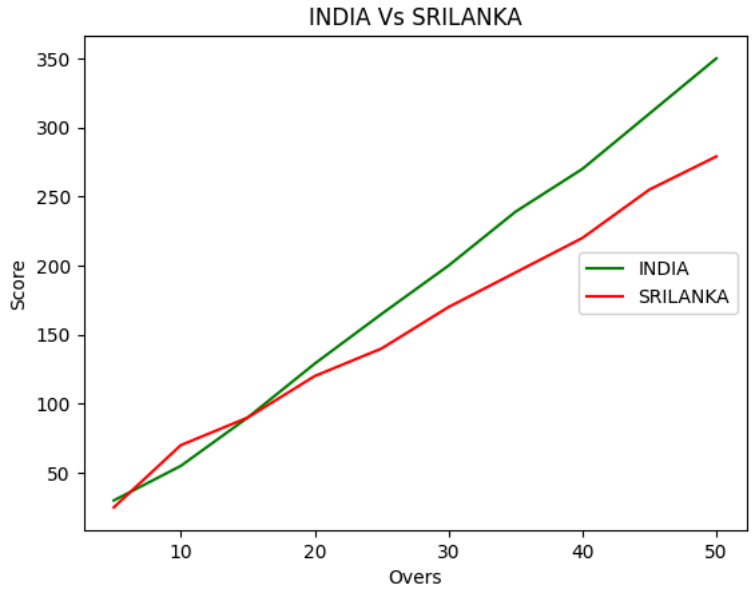
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving india vs srilanka scores.csv to india vs srilanka scores.csv

```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```




```
/tmp/ipykernel_1164/506286342.py:11: UserWarning: No artists with labels found to put in legend.  Note that artists whose la
  cricket.legend()
<matplotlib.legend.Legend at 0x7833cff03c50>
```




```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

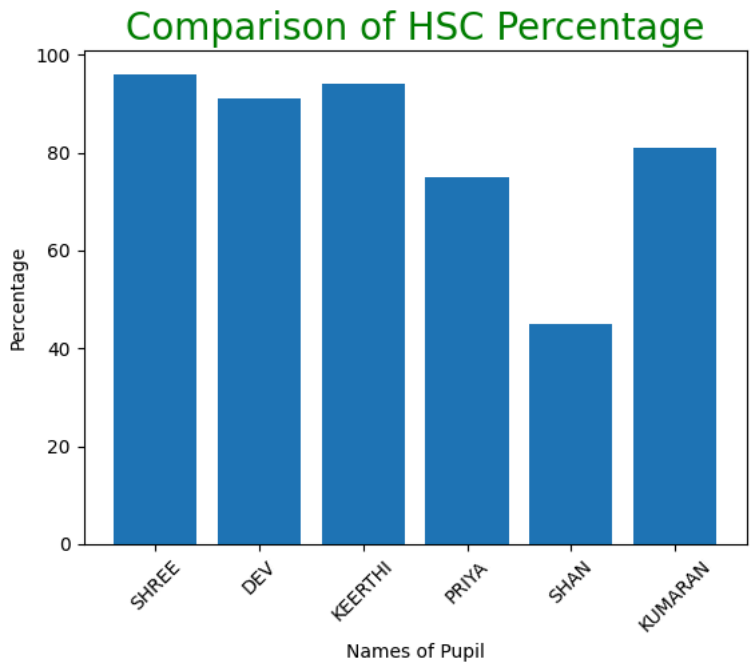
Saving hsc percentage.csv to hsc percentage.csv

```
import pandas as pd
import matplotlib.pyplot as hscmark
import numpy as np

df = pd.read_csv("hsc_percentage.csv")

Names = df["Names"]
xaxis = np.arange(len(Names))
Percentage_hsc = df["Percentage_hsc"]

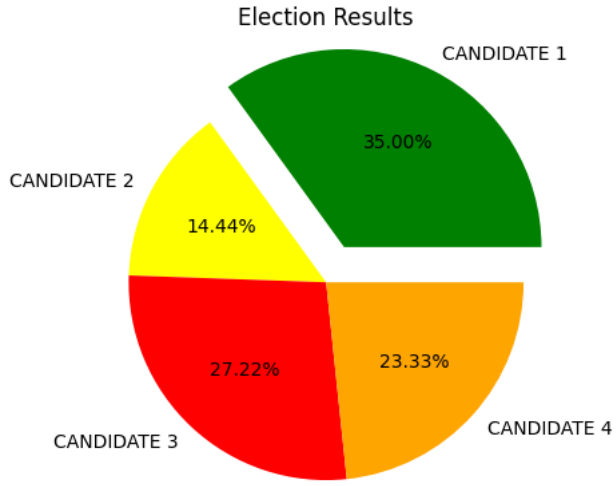
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```




```
import matplotlib.pyplot as election

labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)

election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
election.title('Election Results')
election.show()
```



Exp.no : 08 **LINEAR REGRESSION**

AIM:

Experiment to understand Linear Regression for a given data set.

PYTHON CODE:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df.dropna(inplace=True)
df.info()
df.describe()
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_st
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
model.score(x_train,y_train)
model.score(x_test,y_test)
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
In [27]:
model=pickle.load(open('SalaryPred.model','rb'))
In [28]:
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}: " .format(yr_of_exp,Salary)
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
# Column Non-Null Count Dtype
--- ------ -------------- -----
0 YearsExperience 30 non-null float64
1 Salary 30 non-null int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

	Out[5]: YearsExperience Salary	
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
Max	10.500000	122391.000000

LinearRegression

LinearRegression()
0.9603182547438908
0.9184170849214232
Enter Years of Experience: 44
Estimated Salary for 44.0 years of experience is [[435544.30953887]]:

RESULT:

The code is executed successfully.

Exp.no : 09 **LOGISTIC REGRESSION**

AIM:

Experiment to understand Logistic Regression for a given data set.

PYTHON CODE:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
for i in range(1, 401):
    x_train, x_test, y_train, y_test = train_test_split(
        features, label, test_size=0.2, random_state=i
    )
    model = LogisticRegression()
    model.fit(x_train, y_train)

    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)

    if test_score > train_score:
        print("Test {} Train{} Random State {}".format(
            test_score, train_score, i
        ))


x_train, x_test, y_train, y_test = train_test_split(
    features, label, test_size=0.2, random_state=2
)

finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)


print(finalModel.score(x_train, y_train))
print(finalModel.score(x_test, y_test))


from sklearn.metrics import classification_report

print(classification_report(label, finalModel.predict(features)))
```

OUTPUT:

```
Test 0.6875 Train0.63125 Random State 3
Test 0.7375 Train0.61875 Random State 4
Test 0.6625 Train0.6375 Random State 5
Test 0.65 Train0.640625 Random State 6
Test 0.675 Train0.634375 Random State 7
Test 0.675 Train0.634375 Random State 8
Test 0.65 Train0.640625 Random State 10
Test 0.6625 Train0.6375 Random State 11
Test 0.7125 Train0.625 Random State 13
Test 0.675 Train0.634375 Random State 16
Test 0.7 Train0.628125 Random State 17
Test 0.7 Train0.628125 Random State 21
Test 0.65 Train0.640625 Random State 24
Test 0.6625 Train0.6375 Random State 25
Test 0.75 Train0.615625 Random State 26
Test 0.675 Train0.634375 Random State 27
Test 0.7 Train0.628125 Random State 28
Test 0.6875 Train0.63125 Random State 29
Test 0.6875 Train0.63125 Random State 31
Test 0.6625 Train0.6375 Random State 37
0.834375
0.9125
              precision    recall  f1-score   support

           0       0.85      0.93      0.89       257
           1       0.84      0.71      0.77       143

    accuracy                           0.85       400
   macro avg       0.85      0.82      0.83       400
weighted avg       0.85      0.85      0.85       400
```

RESULT:

The code is executed successfully.

Exp.no : 10 **K-Means ALGORITHM**

AIM:

Experiment to understand K-means clustering algorithm for a given data set.

PYTHON CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans

# Read the dataset
df = pd.read_csv("Mall_Customers.csv")

# Display dataset information
print(df.info())

# Display first 5 rows
print(df.head())

# Pair plot
sns.pairplot(df)
plt.show()

# Select Annual Income and Spending Score
features = df.iloc[:, [3, 4]].values

# Apply K-Means clustering with 5 clusters
model = KMeans(n_clusters=5, n_init=10, random_state=42)
model.fit(features)

# Create final dataframe
Final = df.iloc[:, [3, 4]].copy()

# Predict cluster labels
Final["label"] = model.predict(features)

# Display first 5 rows
print(Final.head())
```

```
# Plot the clusters
sns.set_style("whitegrid")

sns.FacetGrid(
    Final,
    hue="label",
    height=8
).map(
    plt.scatter,
    "Annual Income (k$)",
    "Spending Score (1-100)"
).add_legend()

plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("K-Means Clustering")
plt.show()

# ------------------------------
# Elbow Method
# ------------------------------

features_el = df.iloc[:, [2, 3, 4]].values

wcss = []

for i in range(1, 10):
    model = KMeans(n_clusters=i, n_init=10, random_state=42)
    model.fit(features_el)
    wcss.append(model.inertia_)

# Plot WCSS
plt.plot(range(1, 10), wcss, marker="o")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64

dtypes: int64(4), object(1)
memory usage: 7.9+ KB

First five rows:

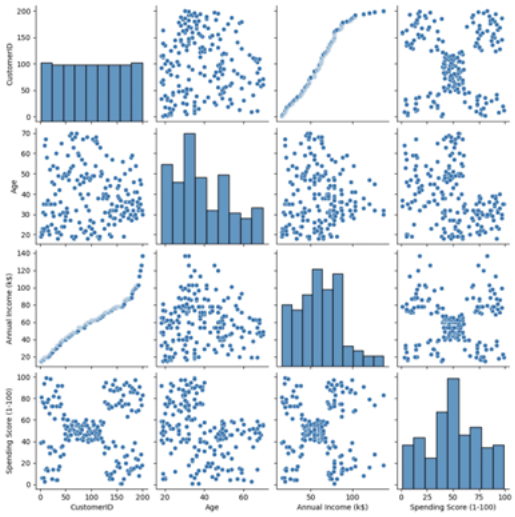
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

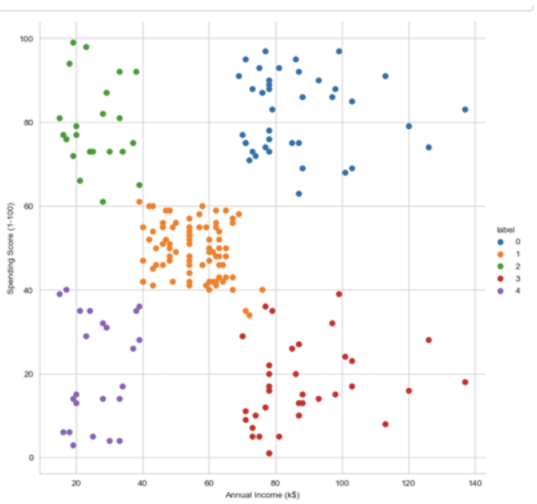
KMeans(n_clusters=5)

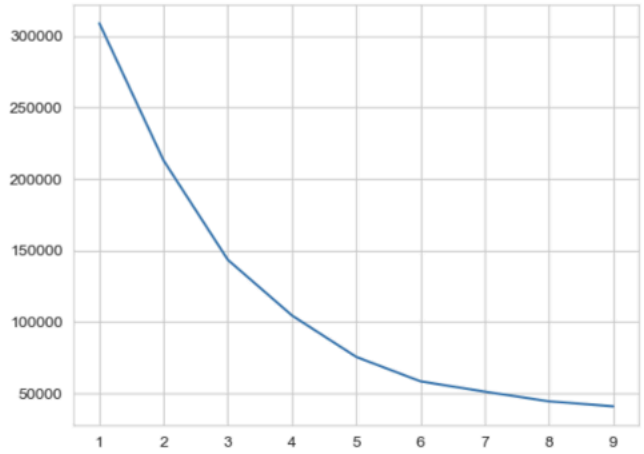
First five clustered records:

   Annual Income (k$)  Spending Score (1-100)  label
0                  15                      39      4
1                  15                      81      2
2                  16                       6      4
3                  16                      77      2
4                  17                      40      4

Plots produced:
1. Pair plot of the dataset
2. K-Means customer-cluster scatter plot
3. Elbow-method WCSS plot for clusters 1 to 9
```







RESULT:

The code is executed successfully.

Exp.no : 11 **KNN ALGORITHM**

AIM:

Experiment to understand KNN algorithm for a given data set

PYTHON CODE:

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report

# Read the dataset
df = pd.read_csv("Iris.csv")

# Display dataset information
df.info()

# Display number of samples in each variety
print(df.variety.value_counts())

# Display first 5 rows
print(df.head())

# Separate features and labels
features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values

# Split data into training and testing sets
xtrain, xtest, ytrain, ytest = train_test_split(
    features,
    label,
    test_size=0.2,
    random_state=42
)

# Create KNN model with 5 neighbors
model_KNN = KNeighborsClassifier(n_neighbors=5)

# Train the model
model_KNN.fit(xtrain, ytrain)
```

```
# Training accuracy
print(model_KNN.score(xtrain, ytrain))

# Testing accuracy
print(model_KNN.score(xtest, ytest))

# Confusion matrix
print(confusion_matrix(label, model_KNN.predict(features)))

# Classification report
print(classification_report(label, model_KNN.predict(features)))
```

OUTPUT:

```
RangeIndex: 150 entries, 0 to 149

Data columns:
sepal.length
sepal.width
petal.length
petal.width
variety
   sepal.length  sepal.width  petal.length  petal.width  variety
0           5.1          3.5           1.4          0.2   Setosa
1           4.9          3.0           1.4          0.2   Setosa
2           4.7          3.2           1.3          0.2   Setosa
3           4.6          3.1           1.5          0.2   Setosa
4           5.0          3.6           1.4          0.2   Setosa
0.9583333333333334
1.0
[[50  0  0]
 [ 0 47  3]
 [ 0  2 48]]
              precision    recall  f1-score   support

Setosa           1.00      1.00      1.00        50
Versicolor       0.96      0.94      0.95        50
Virginica        0.94      0.96      0.95        50

accuracy                             0.97       150
macro avg        0.97      0.97      0.97       150
weighted avg     0.97      0.97      0.97       150
```

RESULT:

The code is executed successfully.

Exp.no : 12 **Z-TEST**

AIM:

A manufacturer claims that the average weight of packets is 50 g.A random sample of 36 packets has
an average weight of 51.2 g with a known σ = 3 g.At a 5% significance level, test the claim.

PYTHON CODE:

```
import numpy as np
from math import sqrt
from scipy.stats import norm
# Given data
x_bar = 51.2 # sample mean
mu_0 = 50 # population mean
sigma = 3 # population standard deviation
n = 36 # sample size
# Calculate Z-statistic
z_stat = (x_bar - mu_0) / (sigma / sqrt(n))
# Two-tailed p-value
p_value = 2 * (1 - norm.cdf(abs(z_stat)))
print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value < alpha:
print("Reject Null Hypothesis → Mean is significantly different from 50 g.")
else:
print("Fail to Reject Null Hypothesis → No significant difference.")
```

OUTPUT:

Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis → Mean is significantly different from 50 g.

RESULT:

The code is executed successfully.

Exp.no : 13

VISUALISATION

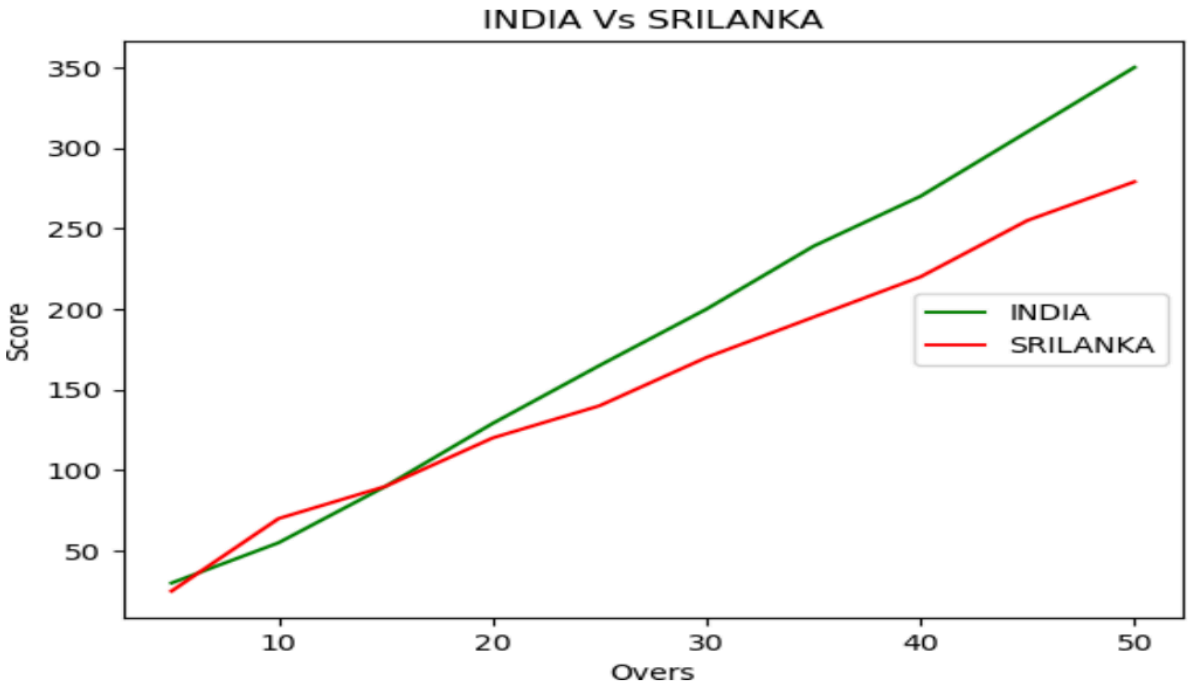
a) AIM:

Conduct an experiment to show data visualization using line plot

PYTHON CODE:

```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```

OUTPUT:



RESULT:

The code is executed successfully.

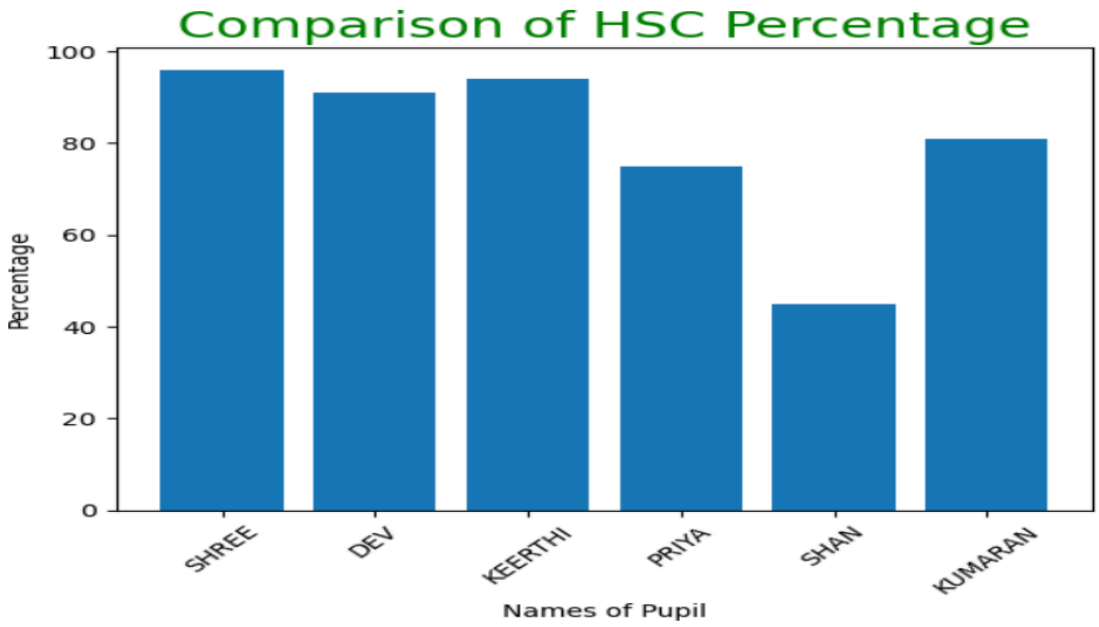
B) AIM:

Conduct an experiment to show data visualization using bar chart

PYTHON CODE:

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

OUTPUT:



RESULT:

The code is executed successfully.

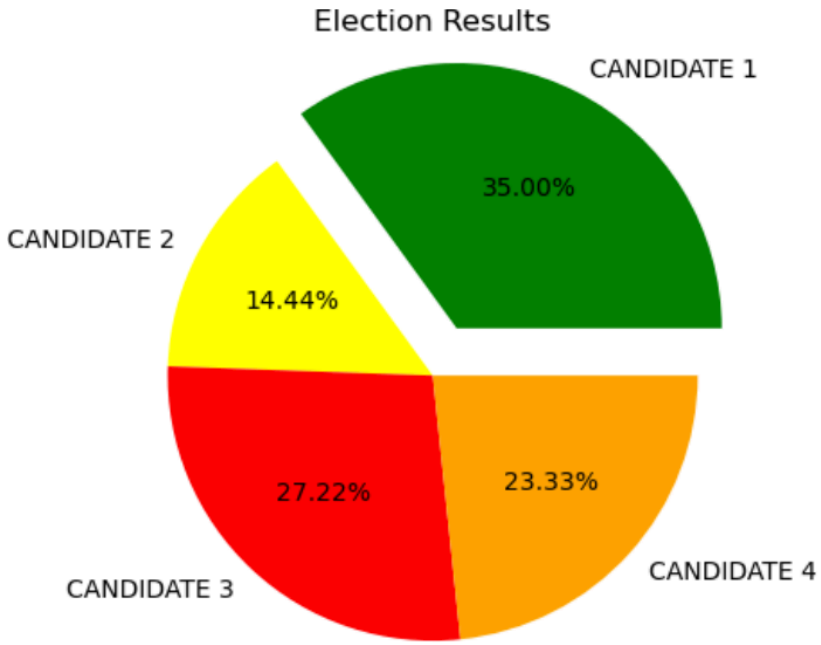
C) AIM:

Conduct an experiment to show data visualization using pie chart.

PYTHON CODE:

```
import matplotlib.pyplot as election
# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
# Plotting the pie chart
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='%0.2f%%')
election.title('Election Results')
election.show()
```

OUTPUT:



RESULT:

The code is executed successfully.

Exp.no : 14

FREQUENCY OF OCCURENCE

a) AIM:

Conduct an experiment to show data visualization using line plot

PYTHON CODE:

```
text = """
Data science is an important field of study.
Data science combines statistics, mathematics,
programming, and machine learning.
Data scientists collect data from different sources.
They clean data, analyze data, and visualize data.
Python is widely used in data science.
Python provides libraries like NumPy, Pandas,
Matplotlib, and Scikit-learn.
These libraries help process large datasets.
Data analysis helps organizations make decisions.
Machine learning uses data to build predictive models.
Data visualization presents information through graphs.
Graphs include line plots, bar charts, and pie charts.
Data mining discovers useful patterns in data.
Big data contains large amounts of information.
Data engineers prepare data for analysis.
Data analysts study trends and summarize results.
Artificial intelligence uses data to learn patterns.
Good data improves the accuracy of predictions.
Data science is useful in healthcare, education,
banking, business, agriculture, and transportation.
Students learn data science to solve real-world problems.
The demand for data science professionals is increasing.
Data processing is an essential step in every project.
"""

word = input("Enter the word to count: ").lower()

words = text.lower().replace(".", "").replace(",", "").split()
frequency = words.count(word)

print("\nTotal words in the text:", len(words))
print("Word searched:", word)
print("Frequency of occurrence:", frequency)
```

OUTPUT:

Enter the word to count: data

Total words in the text: 131
Word searched: data
Frequency of occurrence: 15

RESULT:

The code is executed successfully.

Exp.no : 15 **INITIAL EXPLORATION OF DATA**

AIM:

To collect, load, and perform initial exploration of the diabetes dataset.

PYTHON CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
# Load Excel file
df = pd.read_excel("Diabetes.xlsx")
# Display first five rows
print("First Five Rows:")
print(df.head())

# Display dataset shape
print("\nDataset Shape:")
print(df.shape)
# Display column information
print("\nDataset Information:")
df.info()

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Statistical summary
print("\nStatistical Summary:")
print(df.describe())
# Visualization: Histogram of target values
plt.hist(df["target"], bins=20, edgecolor="black")
plt.title("Distribution of Diabetes Target Values")
plt.xlabel("Target Value")
plt.ylabel("Frequency")
plt.show()
```

OUTPUT:

```
First Five Rows:
        age       sex       bmi  ...        s4        s5        s6
0  0.038076  0.050680  0.061696  ... -0.002592  0.019907 -0.017646
1 -0.001882 -0.044642 -0.051474  ... -0.039493 -0.068332 -0.092204
2  0.085299  0.050680  0.044451  ... -0.002592  0.002861 -0.025930
```

```
3 -0.089063 -0.044642 -0.011595  ...  0.034309  0.022688 -0.009362
4  0.005383 -0.044642 -0.036385  ... -0.002592 -0.031988 -0.046641

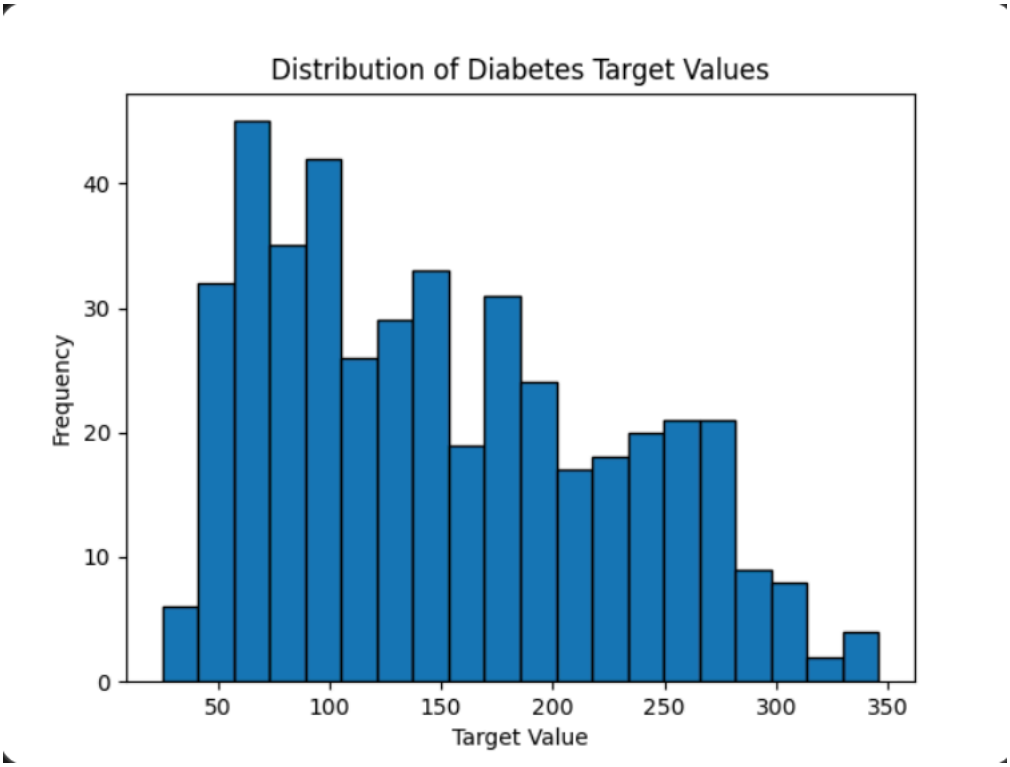
[5 rows x 10 columns]
Dataset Shape:(442, 10)

Column Names:Index(['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3',
's4', 's5', 's6'], dtype='object')

Dataset Information:<class 'pandas.core.frame.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 10 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   age     442 non-null    float64
 1   sex     442 non-null    float64
 2   bmi     442 non-null    float64
 3   bp      442 non-null    float64
 4   s1      442 non-null    float64
 5   s2      442 non-null    float64
 6   s3      442 non-null    float64
 7   s4      442 non-null    float64
 8   s5      442 non-null    float64
 9   s6      442 non-null    float64
dtypes: float64(10)
memory usage: 34.6 KB
None

Missing Values:age    0
sex    0
bmi    0
bp     0
s1     0
s2     0
s3     0
s4     0
s5     0
s6     0
dtype: int64
Statistical Summary:                 age           sex  ...
s5            s6
count  4.420000e+02  4.420000e+02  ...  4.420000e+02  4.420000e+02
mean  -2.511817e-19  1.230790e-17  ...  9.293722e-17  1.130318e-17
std    4.761905e-02  4.761905e-02  ...  4.761905e-02  4.761905e-02
min   -1.072256e-01 -4.464164e-02  ... -1.260971e-01 -1.377672e-01
25%   -3.729927e-02 -4.464164e-02  ... -3.324559e-02 -3.317903e-02
50%    5.383060e-03 -4.464164e-02  ... -1.947171e-03 -1.077698e-03
75%    3.807591e-02  5.068012e-02  ...  3.243232e-02  2.791705e-02
max    1.107267e-01  5.068012e-02  ...  1.335973e-01
1.356118e-01
```

```
[8 rows x 10 columns]
```



RESULT:

The code is executed successfully.

Exp.no : 16 **TIME SERIES ANALYSIS OF WEBSITE TRAFFIC**

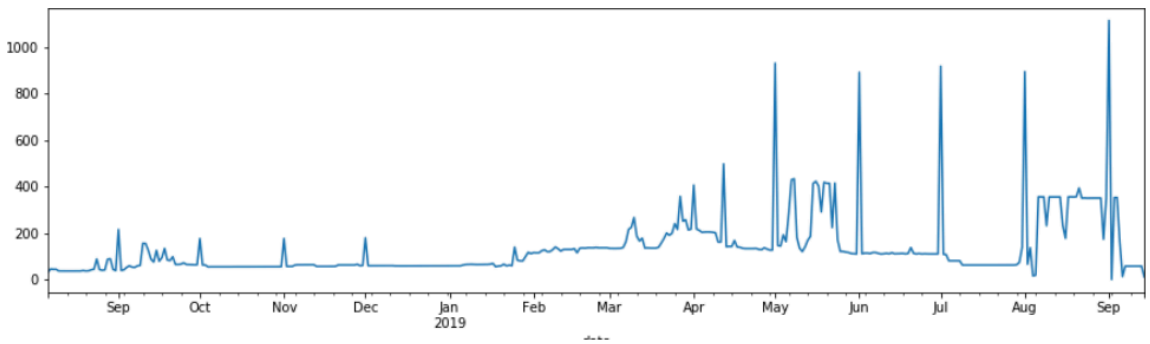
AIM:

To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

PYTHON CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
# Load dataset
data = pd.read_csv('website_traffic.csv', parse_dates=['Date'], index_col='Date')
# Time Series Decomposition
decomposition = seasonal_decompose(data['Page_Views'], model='additive')
decomposition.plot()
22plt.show()
# Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()
# Anomaly Detection
data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) /
data['Page_Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomalies')
plt.legend()
plt.show()
```

OUTPUT:



RESULT:

The code is executed successfully.

Exp.no : 17 **RANDOM SAMPLING AND SAMPLING DISTRIBUTION**

AIM:

To explore random sampling from a population and understand the concept of sampling distribution using Python in Jupyter Notebook.

PYTHON CODE:

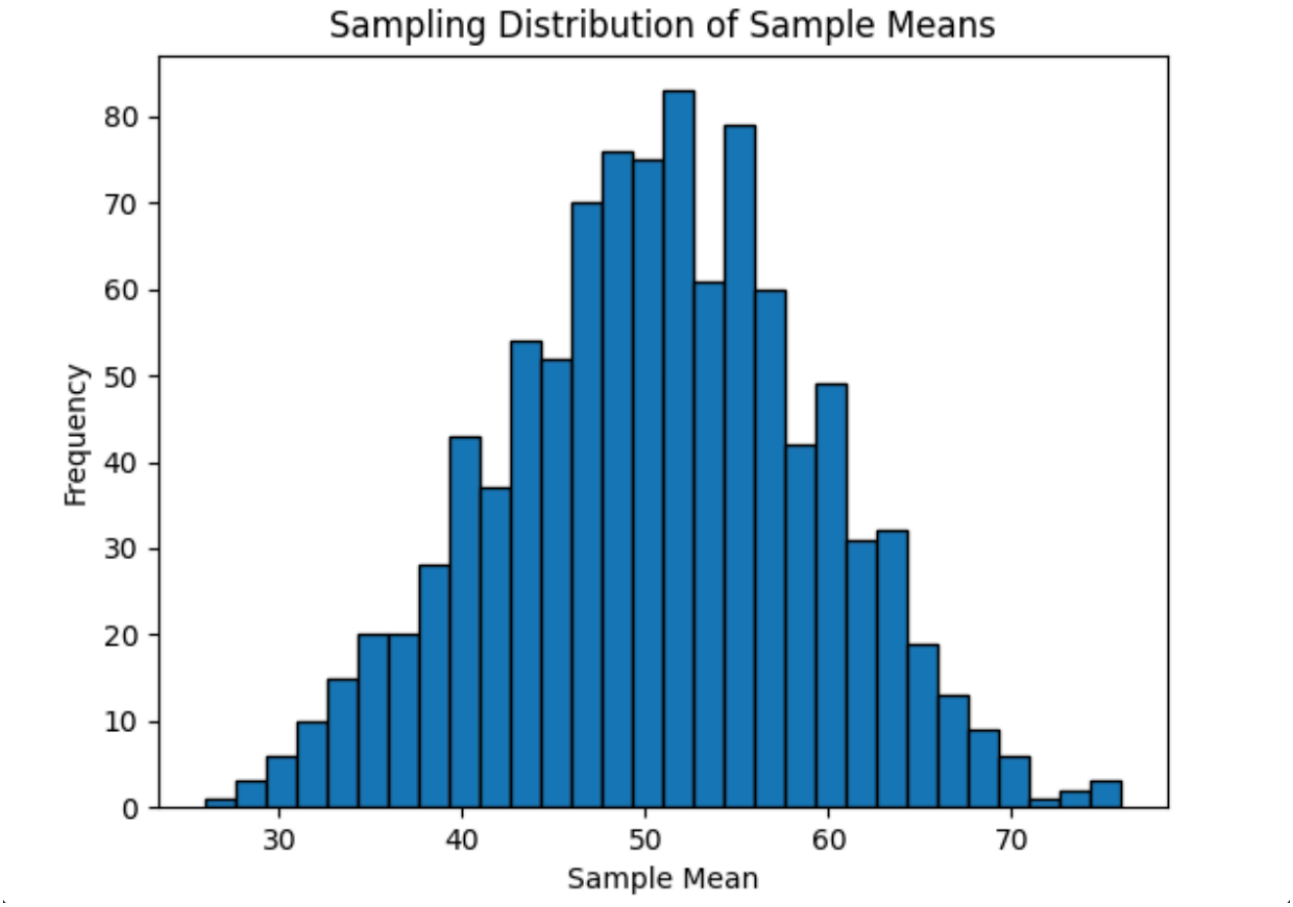
```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
# Load dataset
data = pd.read_csv('website_traffic.csv', parse_dates=['Date'], index_col='Date')
# Time Series Decomposition
decomposition = seasonal_decompose(data['Page_Views'], model='additive')
decomposition.plot()
22plt.show()
# Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()
# Anomaly Detection
data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) /
data['Page_Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomalies')
plt.legend()
plt.show()
```

OUTPUT:

```
Population: [  1   2   3   4   5   6   7   8   9  10  11  12  13
14  15  16  17  18
  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34
35  36
  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52
53  54
  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70
71  72
  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88
89  90
  91  92  93  94  95  96  97  98  99 100]Population Mean: 50.5
```

```
Random Sample: [49 97 55 57 92 87 85 90 77 39]
Sample Mean: 72.8

Mean of Sampling Distribution: 50.692899999999995
```



RESULT:

The code is executed successfully.

Exp.no : 18 **MATPLOTLIB LIBRARY**

AIM:

To experiment to understand Matplotlib library use cases in Data Science through visualization

PYTHON CODE:

```
import matplotlib.pyplot as plt

# Sample dataset
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May',
          'Jun', 'Jul', 'Aug', 'Sep', 'Oct']

sales = [100, 120, 150, 130, 180, 200, 170, 220, 240, 260]
products = ['Laptop', 'Mobile', 'Tablet', 'Watch']
quantity = [30, 45, 20, 25]

# Create multiple visualizations
plt.figure(figsize=(12, 8))

# 1. Line Plot
plt.subplot(2, 2, 1)
plt.plot(months, sales, marker='o')
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Sales')
plt.xticks(rotation=45)

# 2. Bar Chart
plt.subplot(2, 2, 2)
plt.bar(products, quantity)
plt.title('Product Sales')
plt.xlabel('Product')
plt.ylabel('Quantity')

# 3. Pie Chart
plt.subplot(2, 2, 3)
plt.pie(quantity, labels=products, autopct='%1.1f%%')
plt.title('Sales Percentage')

# 4. Histogram
plt.subplot(2, 2, 4)
```

```
plt.hist(sales, bins=5, edgecolor='black')
plt.title('Sales Distribution')
plt.xlabel('Sales')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```

OUTPUT:



RESULT:

The code is executed successfully.

Exp.no : 19 **NUMPY LIBRARY**

AIM:

Understand array function using Numpy library.

PYTHON CODE:

```
import numpy as np

# Create an array
arr = np.array([10, 20, 30, 40, 50])

print("Original Array:", arr)

# Array dimensions
print("Dimensions:", arr.ndim)

# Array shape
print("Shape:", arr.shape)

# Array size
print("Size:", arr.size)

# Sum of array elements
print("Sum:", np.sum(arr))

# Mean of array elements
print("Mean:", np.mean(arr))

# Maximum and minimum values
print("Maximum:", np.max(arr))
print("Minimum:", np.min(arr))

# Sort an array
arr2 = np.array([50, 10, 40, 20, 30])
print("Sorted Array:", np.sort(arr2))

# Reshape an array
arr3 = np.arange(1, 7).reshape(2, 3)
print("Reshaped Array:\n", arr3)

# Array arithmetic
print("Addition:", arr + 5)
```

```
print("Multiplication:", arr * 2)
```

OUTPUT:

```
Original Array: [10 20 30 40 50]
Dimensions: 1
Shape: (5,)
Size: 5
Sum: 150
Mean: 30.0
Maximum: 50
Minimum: 10
Sorted Array: [10 20 30 40 50]
Reshaped Array:
 [[1 2 3]
  [4 5 6]]
Addition: [15 25 35 45 55]
Multiplication: [ 20  40  60  80 100]
```

RESULT:

The code is executed successfully.

Exp.no : 20 **PANDAS LIBRARY**

AIM:

Experiment to understand pandas library use cases in Data science.

PYTHON CODE:

```
import pandas as pd

# Create a dataset
data = {
   'Name': ['Arun', 'Bala', 'Cathy', 'David', 'Eva'],
   'Age': [20, 21, 19, 22, 20],
   'Marks': [85, 78, 92, 67, 88]
}
df = pd.DataFrame(data)
# Display the dataset
print("Original Dataset:")
print(df)
# Display first five rows
print("\nFirst Five Rows:")
print(df.head())
# Display dataset information
print("\nDataset Information:")
print(df.info())
# Calculate statistical summary
print("\nStatistical Summary:")
print(df.describe())

# Filter students with marks above 80
print("\nStudents Scoring Above 80:")
print(df[df['Marks'] > 80])

# Calculate average marks
print("\nAverage Marks:", df['Marks'].mean())
```

OUTPUT:

```
Original Dataset:
    Name  Age  Marks
0   Arun   20     85
```

```
1   Bala   21     78
2  Cathy   19     92
3  David   22     67
4    Eva   20     88
First Five Rows:
    Name  Age  Marks
0   Arun   20     85
1   Bala   21     78
2  Cathy   19     92
3  David   22     67
4    Eva   20     88
Dataset Information:<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    5 non-null      object
 1   Age     5 non-null      int64
 2   Marks   5 non-null      int64
dtypes: int64(2), object(1)
memory usage: 172.0+ bytes
None
Statistical Summary:
             Age      Marks
count   5.000000   5.000000
mean   20.400000  82.000000
std     1.140175   9.823441
min    19.000000  67.000000
25%    20.000000  78.000000
50%    20.000000  85.000000
75%    21.000000  88.000000
max    22.000000  92.000000
Students Scoring Above 80:
    Name  Age  Marks
0   Arun   20     85
2  Cathy   19     92
4    Eva   20     88
Average Marks: 82.0
```

RESULT:

The code is executed successfully.